Chat transcript

Python coding session: all prompts and responses, with code in code blocks

September 27, 2026

User — prompt 1

Ground Rules for this session: Prefer the simplest correct solution; no classes or abstractions unless I ask. Never change existing behavior unless I ask. Use precise Python type hinting for all new functions and methods. Every solution comes with pytest tests. When you're unsure about a requirement, tell me instead of guessing. I'll run the tests myself.

Problem 1. "Add a roster filter"

A class roster is a list of student `dicts`.

```
roster = [
    {"name": "Amara Singh", "grade": 91, "attendance": 0.95},
    {"name": "Liam Chen", "grade": 68, "attendance": 0.72},
    {"name": "Priya Nair", "grade": 84, "attendance": 0.88},
]

def average_grade(roster):
    return sum(s["grade"] for s in roster) / len(roster)
```

Task: Add a function to this code that groups students into "needs attention" (grade < 70 OR attendance < 0.8) and "on track" (everyone else), returning both groups and each group's average grade. Handle an empty roster without crashing. Do not modify `average_grade`.

Challenge:

1. Look closely at the original `average_grade` function. If I pass it an empty list, what happens? How does your solution handle calculating the average for an empty 'needs attention' group without modifying that original function? What average should an empty group report, and what are the tradeoffs of None vs 0 vs omitting it?
2. Add tests for the exact boundaries (grade 70, attendance 0.8) and for each condition triggering alone.

Claude

roster.py

```
from typing import Literal, TypedDict

roster = [
```

```
    {"name": "Amara Singh", "grade": 91, "attendance": 0.95},
    {"name": "Liam Chen", "grade": 68, "attendance": 0.72},
    {"name": "Priya Nair", "grade": 84, "attendance": 0.88},
]

def average_grade(roster):
    return sum(s["grade"] for s in roster) / len(roster)


# --- New code below; nothing above this line was changed. ---

Student = TypedDict("Student", {"name": str, "grade": float, "attendance": float})
GroupName = Literal["needs_attention", "on_track"]


def group_roster(
    roster: list[Student],
) -> dict[GroupName, tuple[list[Student], float | None]]:
    """Split students into "needs_attention" (grade < 70 or attendance < 0.8)
    and "on_track" (everyone else).

    Each group maps to (students, average_grade). An empty group's average is
    None, because the average of zero students is undefined. Input order is
    preserved, and the returned lists hold the same dict objects as the input.
    """
    attention: list[Student] = []
    on_track: list[Student] = []
    for student in roster:
        if student["grade"] < 70 or student["attendance"] < 0.8:
            attention.append(student)
        else:
            on_track.append(student)

    return {
        "needs_attention": (attention, average_grade(attention) if attention else None),
        "on_track": (on_track, average_grade(on_track) if on_track else None),
    }
```

test_roster.py

```
import copy

import pytest

from roster import Student, average_grade, group_roster


def make(name: str, grade: float, attendance: float) -> Student:
    return {"name": name, "grade": grade, "attendance": attendance}


# --- Existing behavior, pinned so nobody "fixes" it by accident ---

def test_average_grade_empty_raises_zero_division() -> None:
    with pytest.raises(ZeroDivisionError):
        average_grade([])


# --- Basic grouping ---

def test_sample_roster() -> None:
    amara = make("Amara Singh", 91, 0.95)
    liam = make("Liam Chen", 68, 0.72)
    priya = make("Priya Nair", 84, 0.88)
```

```
    result = group_roster([amara, liam, priya])

    attention, attention_avg = result["needs_attention"]
    on_track, on_track_avg = result["on_track"]
    assert attention == [liam]
    assert attention_avg == pytest.approx(68)
    assert on_track == [amara, priya]
    assert on_track_avg == pytest.approx(87.5)


def test_empty_roster() -> None:
    result = group_roster([])
    assert result == {"needs_attention": ([], None), "on_track": ([], None)}


def test_everyone_on_track_gives_none_for_empty_attention_group() -> None:
    students = [make("A", 90, 0.9), make("B", 80, 1.0)]
    result = group_roster(students)
    assert result["needs_attention"] == ([], None)
    assert result["on_track"][0] == students
    assert result["on_track"][1] == pytest.approx(85)


def test_everyone_needs_attention_gives_none_for_empty_on_track_group() -> None:
    students = [make("A", 50, 0.9), make("B", 90, 0.5)]
    result = group_roster(students)
    assert result["on_track"] == ([], None)
    assert result["needs_attention"][0] == students
    assert result["needs_attention"][1] == pytest.approx(70)


# --- Exact boundaries: the rule is strict "<", so 70 and 0.8 are on track ---

@pytest.mark.parametrize(
    ("grade", "attendance", "expected_group"),
    [
        (70, 1.0, "on_track"),          # grade exactly at boundary
        (69.99, 1.0, "needs_attention"),  # grade just below
        (69, 1.0, "needs_attention"),
        (100, 0.8, "on_track"),         # attendance exactly at boundary
        (100, 0.7999, "needs_attention"),  # attendance just below
        (100, 0.79, "needs_attention"),
        (70, 0.8, "on_track"),          # both exactly at boundary
    ],
)
def test_boundaries(grade: float, attendance: float, expected_group: str) -> None:
    student = make("X", grade, attendance)
    result = group_roster([student])
    other_group = "on_track" if expected_group == "needs_attention" else "needs_attention"
    assert result[expected_group] == ([student], pytest.approx(grade))  # type: ignore[index]
    assert result[other_group] == ([], None)  # type: ignore[index]


# --- Each condition triggering alone, and both together ---

def test_low_grade_alone_triggers_attention() -> None:
    student = make("X", 55, 1.0)
    assert group_roster([student])["needs_attention"][0] == [student]


def test_low_attendance_alone_triggers_attention() -> None:
    student = make("X", 100, 0.5)
    assert group_roster([student])["needs_attention"][0] == [student]
```

```
def test_both_conditions_trigger_attention_once() -> None:
    student = make("X", 40, 0.3)
    result = group_roster([student])
    assert result["needs_attention"][0] == [student]
    assert result["on_track"][0] == []


# --- Side effects ---

def test_preserves_input_order() -> None:
    students = [make("C", 50, 1.0), make("A", 90, 1.0), make("B", 60, 1.0)]
    result = group_roster(students)
    assert [s["name"] for s in result["needs_attention"][0]] == ["C", "B"]


def test_does_not_mutate_input() -> None:
    students = [make("A", 50, 1.0), make("B", 90, 1.0)]
    snapshot = copy.deepcopy(students)
    group_roster(students)
    assert students == snapshot
```

1. What `average_grade([])` **does.** `sum()` of an empty generator is `0` and `len([])` is `0`, so it evaluates `0 / 0` and raises `ZeroDivisionError`. I left it as is and added a test that pins that behavior, so nobody "fixes" it by accident. `group_roster` avoids the crash by calling `average_grade` only when a group has students (`average_grade(attention) if attention else None`). That way the averaging logic stays in one place and isn't duplicated.

What an empty group should report. I chose `None`. Here are the tradeoffs:

- `None` is honest, because the average of zero students is undefined. The return type is `float | None`, so a type checker will force callers to handle the empty case. The cost is that callers have to check for it, and something like `f"{avg:.1f}"` will crash on `None`.
- `0` is easy to print, but it's wrong. You can't tell it apart from a group that really averages 0, and it quietly skews any later math, such as averaging the group averages. An empty "needs attention" group would look like a failing group.
- **Omitting the key** makes the result's shape depend on the data. Callers would get `KeyError`s or need `.get()` everywhere, and it's harder to type precisely.
- (`float('nan')` is another option. I'd avoid it because it spreads silently through calculations and `nan != nan` breaks equality checks.)

2. Boundaries. Your spec says `grade < 70` and `attendance < 0.8`, so I read those as strict: exactly 70 and exactly 0.8 count as **on track**. The parametrized test covers both values exactly, values just below each one, both at the boundary together, and each condition triggering alone.

Things I wasn't sure about, and how I handled each:

- **Type precision vs. "no classes."** A plain `dict[str, ...]` can't precisely type a mix of `str` and `float` values. So I used the *functional* `TypedDict(...)` form for `Student`. It's only a type annotation, with no runtime behavior. If you'd rather avoid it completely, I can switch to a less precise alias.
- **The existing module-level** `roster` **isn't annotated,** so mypy infers `list[dict[str, object]]` for it. Calling `group_roster(roster)` on it will fail a type check even though it runs fine. Fixing that means annotating an existing line, which I didn't do without your go-ahead.
- **Floating-point attendance.** Attendance values computed by accumulating floats could land at something like `0.7999999999` and get flagged. The literal `0.8` works correctly. Tell me if these values come from calculations and you want a tolerance.

- **The returned lists hold the same dict objects as the input,** not copies. Mutating one will affect the original roster.
- **Missing or malformed keys** aren't validated; they raise `KeyError` or `TypeError`. The code needs **Python 3.10+** for the `float | None` syntax.

User — prompt 2

write a pytest suite for this grade converter. make sure to test the exact decimal boundaries (like 89.9 vs 90).

important: do NOT modify the function itself to fix anything. just write tests to lock in the current behavior. if the current code crashes on certain inputs, use pytest.raises to document that crash.

```
def letter_grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"
```

Claude

grades.py

```
def letter_grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"
```

test_grades.py

```
"""Characterization tests: these lock in letter_grade's CURRENT behavior,
including behavior that may be undesirable (no range validation, NaN -> "F",
bools accepted). A failing test here means behavior changed, not that the
change is wrong -- update the test deliberately if the change was intended.
"""
import math
from decimal import Decimal
from fractions import Fraction

import pytest

from grades import letter_grade

BOUNDARIES = [(90, "A", "B"), (80, "B", "C"), (70, "C", "D"), (60, "D", "F")]


# --- Exact decimal boundaries ------------------------------------------------

@pytest.mark.parametrize(
    ("score", "expected"),
```

```
    [
        (90, "A"), (90.0, "A"), (89.9, "B"), (89.99, "B"),
        (80, "B"), (80.0, "B"), (79.9, "C"), (79.99, "C"),
        (70, "C"), (70.0, "C"), (69.9, "D"), (69.99, "D"),
        (60, "D"), (60.0, "D"), (59.9, "F"), (59.99, "F"),
    ],
)
def test_decimal_boundaries(score: float, expected: str) -> None:
    assert letter_grade(score) == expected


@pytest.mark.parametrize(("cutoff", "at_or_above", "below"), BOUNDARIES)
def test_largest_float_below_cutoff_gets_lower_grade(
    cutoff: int, at_or_above: str, below: str
) -> None:
    # math.nextafter gives the closest representable float just under the cutoff.
    just_below = math.nextafter(cutoff, 0)
    assert just_below < cutoff
    assert letter_grade(just_below) == below
    assert letter_grade(cutoff) == at_or_above


@pytest.mark.parametrize(("cutoff", "at_or_above", "below"), BOUNDARIES)
def test_smallest_float_above_cutoff_gets_same_grade(
    cutoff: int, at_or_above: str, below: str
) -> None:
    assert letter_grade(math.nextafter(cutoff, 100)) == at_or_above


def test_float_literal_that_rounds_up_to_cutoff_gets_higher_grade() -> None:
    # This literal has more precision than a float can hold; Python stores it as 90.0.
    score = 89.999999999999999
    assert score == 90.0
    assert letter_grade(score) == "A"


# --- Interior values -------------------------------------------------------

@pytest.mark.parametrize(
    ("score", "expected"),
    [(95, "A"), (85, "B"), (75, "C"), (65, "D"), (30, "F"), (0, "F"), (100, "A")],
)
def test_mid_band_scores(score: int, expected: str) -> None:
    assert letter_grade(score) == expected


# --- No range validation: out-of-range values are graded, not rejected -------

@pytest.mark.parametrize(
    ("score", "expected"),
    [(100.1, "A"), (150, "A"), (10**9, "A"), (-0.1, "F"), (-50, "F")],
)
def test_out_of_range_scores_are_not_rejected(score: float, expected: str) -> None:
    assert letter_grade(score) == expected


def test_positive_infinity_is_a() -> None:
    assert letter_grade(math.inf) == "A"


def test_negative_infinity_is_f() -> None:
    assert letter_grade(-math.inf) == "F"


def test_nan_falls_through_to_f() -> None:
```

```
    # Every comparison with NaN is False, so it reaches the else branch.
    assert letter_grade(math.nan) == "F"


# --- Other numeric types that compare with int ------------------------------

@pytest.mark.parametrize(
    ("score", "expected"),
    [
        (Decimal("90"), "A"),
        (Decimal("89.9"), "B"),
        (Decimal("89.999999999999999999"), "B"),  # Decimal keeps the precision a float loses
        (Decimal("60.0"), "D"),
        (Decimal("59.99"), "F"),
        (Fraction(180, 2), "A"),
        (Fraction(179, 2), "B"),
    ],
)
def test_decimal_and_fraction_inputs(score: Decimal | Fraction, expected: str) -> None:
    assert letter_grade(score) == expected


@pytest.mark.parametrize("score", [True, False])
def test_bools_are_accepted_as_numbers(score: bool) -> None:
    # bool is a subclass of int (True == 1, False == 0), so both grade as "F".
    assert letter_grade(score) == "F"


# --- Inputs that currently crash --------------------------------------------

@pytest.mark.parametrize(
    "score",
    [None, "95", "", [90], (90,), {"score": 90}, 90 + 0j],
    ids=["None", "numeric-string", "empty-string", "list", "tuple", "dict", "complex"],
)
def test_non_comparable_inputs_raise_type_error(score: object) -> None:
    with pytest.raises(TypeError, match="not supported between instances"):
        letter_grade(score)
```

The suite is in `test_grades.py`. I copied the function unchanged into `grades.py` so the tests have something to import.

Boundaries. Each cutoff (90, 80, 70, 60) is tested at the exact value, at the value with a trailing `.0`, and at `.9` and `.99` below it. Two more checks use `math.nextafter` to get the float immediately below and immediately above each cutoff, which is the tightest boundary a float can express.

One test locks in a float trap: the literal `89.999999999999999` is stored as exactly `90.0`, so it gets an "A". The same value written as a `Decimal` keeps its precision and gets a "B", and a separate test covers that.

Current behavior these tests lock in, which may or may not be what you want:

- **No range validation.** `150`, `10**9` and `inf` all get "A", and negative scores get "F".
- `NaN` **returns "F" instead of crashing.** Every comparison with NaN is `False`, so it falls through to the `else` branch.
- **Bools are accepted.** `True` and `False` compare as `1` and `0`, so both get "F".
- **Strings crash rather than being converted.** `"95"` raises `TypeError`, as do `None`, lists, tuples, dicts and complex numbers. I documented these with `pytest.raises(TypeError, match="not supported between instances")`. The `match` part pins the reason for the crash, not only its type.

Things I assumed:

- **Module name.** I assumed the function lives in a module called `grades`. If yours is named differently, change the import line.
- **No rounding before grading.** A score of `89.95` gets "B", even though a report might display it as "90.0". The tests lock that in. If the displayed score is supposed to decide the grade, that's a behavior change, and these tests would fail on purpose.
- **Python version.** You need Python 3.10+ for the `Decimal | Fraction` type hint. `math.nextafter` needs 3.9+.

I haven't run the tests, per your ground rules.

User — prompt 3

the tests look good, but looking at the function, it handles weird data pretty poorly (like negative numbers, scores over 100, or passing a string).

give me 2 or 3 quick "clarifying questions" I should ask the product owner to figure out how we actually want to handle those edge cases before I refactor the code.

Claude

1. **What's the valid score range, and what should happen outside it?** Is it strictly 0–100, or can extra credit push a score above 100? For an out-of-range score, should we reject it with an error, clamp it (150 → 100, -5 → 0), or grade it as-is like we do today? This decides whether a negative number means "bad data, stop" or "just an F."
2. **Should scores that aren't numbers ever be accepted?** Some sources, such as CSV imports or form fields, may send `"95"` as text. Should we convert numeric strings, or reject anything that isn't already a number? And what about a missing score (`None`) or `NaN`: is that an error, or its own result like "Incomplete" rather than a silent "F"?
3. **Is the grade based on the exact score or the rounded one?** If a report shows `89.95` as "90.0", should the student get the "A" they see or the "B" the raw number gives? And if rounding applies, to how many decimal places?

The third question isn't about bad data, but it's the one most likely to generate complaints from students, and your boundary tests will need to change depending on the answer.

User — prompt 4

```
def import_teacher_row(row):
    name = row[0].strip()
    email = row[1].strip().lower()
    if not name or not email:
        raise ValueError("missing required field")
    return {"name": name, "email": email, "role": "teacher"}

def import_student_row(row):
    name = row[0].strip()
    email = row[1].strip().lower()
    if not name or not email:
        raise ValueError("missing required field")
    return {"name": name, "email": email, "role": "student", "grade_level": row[2]}
```

refactor these to remove the duplication

crucial rule: do not change the behavior at all. if the original threw a specific exception on a specific input, the new one has to do exactly the same thing. add a third importer for administrators (role: "administrator", no grade_level) using whatever new structure you build. use strict python typing.

before you write the refactored code, write a parameterized pytest suite that imports the ORIGINAL functions to use as the source of truth. run these exact inputs through both versions: `['', 'a@x.com']`, `['Ada']`, `[]`, `[None, 'a@x.com']`, and `['Ada', 'b@x.com', '9']`. assert that the new version returns the exact same thing OR raises the exact same exception type and message as the original.

spit out the tests first, then the refactored code.

Claude

Here are the tests first. They import the original functions from `importers_original.py`, which holds your code copied unchanged, and treat them as the source of truth.

test_importers.py

```
"""Equivalence tests: the ORIGINAL importers are the source of truth.

For every input, the refactored function must either return exactly what the
original returns (same keys, same order, same values) or raise the exact same
exception type with the exact same message.
"""
import copy
from collections.abc import Callable, Mapping
from typing import Any

import pytest

import importers as refactored
import importers_original as original

REQUIRED_INPUTS = [
    pytest.param(["", "a@x.com"], id="empty-name"),
    pytest.param(["Ada"], id="missing-email"),
    pytest.param([], id="empty-row"),
    pytest.param([None, "a@x.com"], id="none-name"),
    pytest.param(["Ada", "b@x.com", "9"], id="valid-with-grade"),
]

# Extra inputs chosen to catch evaluation-order mistakes in a refactor.
EXTRA_INPUTS = [
```

```
    pytest.param(["Ada", ""], id="empty-email"),
    pytest.param(["   ", "a@x.com"], id="whitespace-name"),
    pytest.param(["", None], id="empty-name-then-none-email"),  # AttributeError must win over ValueError
    pytest.param(["Ada", "b@x.com"], id="valid-without-grade"),  # student: IndexError only after
        validation
    pytest.param(["", "a@x.com"], id="invalid-without-grade"),  # student: ValueError, not IndexError
    pytest.param([" Ada ", " B@X.COM ", " 9 "], id="whitespace-and-case"),  # grade_level is not stripped
    pytest.param(("Ada", "b@x.com", "9"), id="tuple-row"),
    pytest.param(("Ada",), id="tuple-row-too-short"),  # message says "tuple", not "list"
]

ALL_INPUTS = REQUIRED_INPUTS + EXTRA_INPUTS

PAIRS = [
    pytest.param(original.import_teacher_row, refactored.import_teacher_row, id="teacher"),
    pytest.param(original.import_student_row, refactored.import_student_row, id="student"),
]


def run(func: Callable[[Any], Mapping[str, object]], row: Any) -> tuple[str, Any]:
    """Capture a function's full observable outcome on a fresh copy of row."""
    try:
        result = func(copy.copy(row))
    except Exception as exc:
        return ("raised", (type(exc), str(exc)))
    return ("returned", list(result.items()))  # list of items also pins key order


@pytest.mark.parametrize("row", ALL_INPUTS)
@pytest.mark.parametrize(("old", "new"), PAIRS)
def test_refactor_matches_original(
    old: Callable[[Any], Mapping[str, object]],
    new: Callable[[Any], Mapping[str, object]],
    row: Any,
) -> None:
    assert run(new, row) == run(old, row)


@pytest.mark.parametrize("row", ALL_INPUTS)
def test_administrator_matches_original_teacher_except_role(row: Any) -> None:
    kind, value = run(original.import_teacher_row, row)
    if kind == "returned":
        value = [(k, "administrator" if k == "role" else v) for k, v in value]
    assert run(refactored.import_administrator_row, row) == (kind, value)
```

importers_original.py

```
def import_teacher_row(row):
    name = row[0].strip()
    email = row[1].strip().lower()
    if not name or not email:
        raise ValueError("missing required field")
    return {"name": name, "email": email, "role": "teacher"}

def import_student_row(row):
    name = row[0].strip()
    email = row[1].strip().lower()
    if not name or not email:
        raise ValueError("missing required field")
    return {"name": name, "email": email, "role": "student", "grade_level": row[2]}
```

Now the refactored code:

importers.py

```
from collections.abc import Sequence
from typing import Literal, TypedDict

TeacherRecord = TypedDict(
    "TeacherRecord",
    {"name": str, "email": str, "role": Literal["teacher"]},
)
StudentRecord = TypedDict(
    "StudentRecord",
    {"name": str, "email": str, "role": Literal["student"], "grade_level": str},
)
AdministratorRecord = TypedDict(
    "AdministratorRecord",
    {"name": str, "email": str, "role": Literal["administrator"]},
)


def _name_and_email(row: Sequence[str]) -> tuple[str, str]:
    """Parse and validate the fields every importer shares.

    The order of these steps is part of the behavior: index errors and
    AttributeErrors from row[0], then row[1], come before the ValueError.
    """
    name = row[0].strip()
    email = row[1].strip().lower()
    if not name or not email:
        raise ValueError("missing required field")
    return name, email


def import_teacher_row(row: Sequence[str]) -> TeacherRecord:
    name, email = _name_and_email(row)
    return {"name": name, "email": email, "role": "teacher"}


def import_student_row(row: Sequence[str]) -> StudentRecord:
    name, email = _name_and_email(row)
    # row[2] is read only after validation, matching the original.
    return {"name": name, "email": email, "role": "student", "grade_level": row[2]}


def import_administrator_row(row: Sequence[str]) -> AdministratorRecord:
    name, email = _name_and_email(row)
    return {"name": name, "email": email, "role": "administrator"}
```

What your five inputs do in the original code, so you know what the tests expect before you run them:

Input	Teacher	Student
`['', 'a@x.com']`	`ValueError("missing required field")`	same `ValueError`; it never reaches `row[2]`
`['Ada']`	`IndexError("list index out of range")`	same
`[]`	`IndexError("list index out of range")`	same
`[None, 'a@x.com']`	`AttributeError` (`None` has no `.strip`)	same

Input	Teacher	Student
`['Ada', 'b@x.com', '9']`	teacher dict; the extra column is ignored	student dict with `grade_level '9'`

How the tests compare. The comparison uses `type(exc)`, not `isinstance`, so a subclass of the right exception won't pass. It also checks the returned dict as a list of items, which pins key order as well as the values. Each call gets a fresh copy of the row.

Why the refactor preserves behavior. The shared helper runs the same steps in the same order: index into `row[0]`, strip it, index into `row[1]`, strip and lowercase it, then validate. The student importer still reads `row[2]` only after validation.

Extra inputs. I added eight inputs on top of your five, aimed at that ordering. For example, `['', None]` must raise `AttributeError`, not `ValueError`, and a too-short tuple must say "tuple index" in its message.

Things to flag:

- **Administrator behavior is my assumption.** I made it identical to teacher (same stripping, lowercasing and validation, and extra columns ignored), and its test enforces that by using the original teacher function as the reference. If admins need something different, such as a required domain or an extra field, tell me.
- **Tracebacks differ.** Exceptions now come from inside `_name_and_email`, so there's an extra stack frame. The exception type and message are identical. Only code that inspects tracebacks would notice.
- **About the typing.** I used `Sequence[str]` because the code only needs indexing. One gotcha: a plain `str` also counts as a `Sequence[str]`, so mypy won't catch `import_teacher_row("Ada")`. If your rows always come from `csv.reader`, `list[str]` is stricter; tell me if you want that.
- **Type hints don't change runtime behavior.** A `None` in the row still raises `AttributeError` exactly as before.
- `importers_original.py` **is untyped on purpose,** since it's your code copied unchanged. Under `mypy --strict`, it will report missing annotations in that file only.

I haven't run the tests.

User — prompt 5

for this, don't write any code yet.

i need an in-memory rate limiter with a single method: `allow(client_id: str) -> bool`. it has to enforce at most N requests per rolling W-second window for a single process.

the spec requires that a second strategy (like fixed window or token bucket) can be plugged in later without changing how callers use `allow()`.

before coding, pitch me 2 or 3 design options for structuring this so strategies are swappable. compare tradeoffs in memory, accuracy, and complexity. keep our ground rules in mind: simplest correct solution, no bloated abstract base class hierarchies or needless abstractions.

Claude

A quick note first: at this level, structure and algorithm are separate choices. How you organize swappable strategies has almost no effect on memory or accuracy; those come from the algorithm inside each strategy. So below I compare the structures on complexity and fit, then cover the algorithms separately.

Structural options

Option A: factory functions that return a closure. Each strategy is a function like `sliding_window_limiter(n, w)` that returns an `allow(client_id) -> bool` function holding its state privately. Anything of type `Callable[[str], bool]` is swappable, so callers just call `allow("abc")`.

- *Pros:* no classes at all, which fits your ground rules most literally, and it's the least code.
- *Cons:* your spec says *method*, and this is a bare function. The state is hidden, so tests can't inspect it, and adding a second operation later (`reset`, stats, cleanup) is awkward.

Option B: one small class per strategy, with a `Protocol` for the type. You'd write `SlidingWindowLimiter` now and `TokenBucketLimiter` later, each with an `allow` method. There's no base class and no inheritance. A three-line `typing.Protocol` describes "anything with `allow(str) -> bool`", and callers annotate against that. Swapping means constructing a different class; call sites don't change.

- *Pros:* this matches "a method" exactly, and each strategy is self-contained and easy to test. The Protocol exists only for type checking, so it adds no runtime machinery.
- *Cons:* it requires classes, which your ground rules say to avoid unless you ask. I'd read the spec's "method" as that ask, but that's your call.

Option C: one `RateLimiter` class that wraps a pluggable strategy function. The wrapper owns the shared concerns (the per-client state dictionary, locking, idle-client eviction) and delegates the "allow?" decision to a strategy function.

- *Pros:* cross-cutting code is written once, which pays off if you end up with three or more strategies and need thread safety.
- *Cons:* strategies need different per-client state (a list of timestamps vs. a token count plus a timestamp), so the wrapper has to be generic over that state. That's real complexity for a benefit you don't have yet. This is the option most at risk of being needless abstraction.

My recommendation: B. It's the simplest design that literally satisfies "a method" and "swappable without changing callers." If you only have two strategies, the shared code you'd consolidate with C is a few lines. If a plain function satisfies the spec, A is even simpler.

Algorithms, where memory and accuracy actually differ

Algorithm	Memory per client	Accuracy vs. "N per rolling W seconds"	Complexity
Sliding log (a deque of timestamps)	Up to N timestamps	Exact	Low
Sliding window counter (weights the previous window's count)	2 counters	Approximate; assumes requests were evenly spread	Low–medium
Fixed window	1 counter + window start	Up to 2N in a short burst around a window boundary	Lowest
Token bucket	Token count + last refill time	Different rule: allows bursts up to capacity, then enforces a steady rate	Low

Your requirement is an exact rolling window, so the first strategy should be the **sliding log**. Its cost is O(N) memory per active client. That's fine for modest N, but it gets expensive if N is in the thousands and you have many clients.

Questions before I code

1. **Concurrency:** Will `allow()` be called from multiple threads or from asyncio tasks? One process can still have races, and that decides whether we need a lock.
2. **Unbounded client IDs:** Should idle clients be removed from memory? Without that, memory grows with every client ID ever seen, which matters if IDs come from something like IP addresses.
3. **Do denied requests count?** The usual answer is no, so a client that keeps hammering still gets through once the window clears. If denied requests count, a persistent client could lock itself out indefinitely.
4. **Window edge:** Is a request made exactly W seconds ago still inside the window? I'd say no, meaning the window is (now − W, now], but it should be decided and tested.
5. **Clock:** I plan to use `time.monotonic` and inject it for tests, so tests don't sleep. Is that okay, or do you need wall-clock time?
6. **Scope of the limit:** Does every client share the same N and W, or can limits vary per client?

User — prompt 6

let's go with option B and the sliding log algorithm. that seems the most robust for exact windows.

to answer your questions:

1. single thread, synchronous. no locks needed.
2. yes, unbounded clients! millions of one-time clients pinging the system means you must clean up idle ones. build eviction into `allow()`.
3. denied requests do not count against the limit.
4. exactly W seconds ago is OUTSIDE the window. strict < check.
5. time.monotonic is perfect. use dependency injection for the time function so tests can simulate time passing without time.sleep().
6. limits (N and W) are global for the instance, not per-client.

implement it. requirements:

- use strict python typing (e.g. `Protocol`, `deque`, `Callable`).
- build the `SlidingWindowLimiter` class.
- write the `pytest` suite. include tests that prove time-injection works, prove that a 6th request in a 5-request window is denied, prove exactly-W-seconds boundaries, and crucially, write a test that proves memory is actually reclaimed when old clients age out.

output the tests first, then the implementation.

Claude

Tests first:

test_rate_limiter.py

```
import gc
import inspect
import time
import weakref
from collections.abc import Callable

import pytest

from rate_limiter import RateLimiter, SlidingWindowLimiter

LIMIT = 5
WINDOW = 10.0


def make_clock(start: float = 0.0) -> tuple[Callable[[], float], Callable[[float], None]]:
    """A fake clock: returns (clock, set_time). Time only moves when set_time is called."""
    current = [start]

    def clock() -> float:
        return current[0]

    def set_time(t: float) -> None:
        current[0] = t

    return clock, set_time


def make_limiter(
```

```
    limit: int = LIMIT, window: float = WINDOW, start: float = 0.0
) -> tuple[SlidingWindowLimiter, Callable[[float], None]]:
    clock, set_time = make_clock(start)
    return SlidingWindowLimiter(limit, window, clock=clock), set_time


# --- Time injection ------------------------------------------------------------

def test_default_clock_is_time_monotonic() -> None:
    default = inspect.signature(SlidingWindowLimiter).parameters["clock"].default
    assert default is time.monotonic


def test_default_clock_works_without_injection() -> None:
    limiter = SlidingWindowLimiter(1, WINDOW)
    assert limiter.allow("a") is True
    assert limiter.allow("a") is False


def test_frozen_injected_clock_keeps_denying() -> None:
    limiter, _ = make_limiter(limit=1)
    assert limiter.allow("a") is True
    for _ in range(100):
        assert limiter.allow("a") is False  # no real time matters, only the fake clock


def test_advancing_injected_clock_restores_access_without_sleeping() -> None:
    limiter, set_time = make_limiter(limit=1)
    assert limiter.allow("a") is True
    set_time(WINDOW)
    assert limiter.allow("a") is True


def test_clock_is_consulted_once_per_call() -> None:
    calls: list[None] = []

    def counting_clock() -> float:
        calls.append(None)
        return 0.0

    limiter = SlidingWindowLimiter(LIMIT, WINDOW, clock=counting_clock)
    for _ in range(7):
        limiter.allow("a")
    assert len(calls) == 7


# --- The limit -----------------------------------------------------------------

def test_sixth_request_in_five_request_window_is_denied() -> None:
    limiter, set_time = make_limiter()
    for t in range(5):
        set_time(float(t))
        assert limiter.allow("a") is True
    set_time(4.5)
    assert limiter.allow("a") is False


def test_limit_is_per_client() -> None:
    limiter, _ = make_limiter()
    for _ in range(LIMIT):
        assert limiter.allow("a") is True
    assert limiter.allow("a") is False
    assert limiter.allow("b") is True
```

```
def test_window_is_rolling_not_fixed() -> None:
    limiter, set_time = make_limiter(limit=2)
    set_time(0.0)
    assert limiter.allow("a") is True
    set_time(5.0)
    assert limiter.allow("a") is True
    set_time(9.0)
    assert limiter.allow("a") is False   # 0 and 5 still in window
    set_time(10.0)
    assert limiter.allow("a") is True    # 0 aged out; window now holds 5, 10
    set_time(14.0)
    assert limiter.allow("a") is False   # 5 and 10 still in window
    set_time(15.0)
    assert limiter.allow("a") is True    # 5 aged out


# --- Exactly-W-seconds boundaries (exactly W ago is OUTSIDE the window) -------

def test_request_just_under_w_seconds_later_is_denied() -> None:
    limiter, set_time = make_limiter(limit=1)
    assert limiter.allow("a") is True
    set_time(9.999)
    assert limiter.allow("a") is False


def test_request_exactly_w_seconds_later_is_allowed() -> None:
    limiter, set_time = make_limiter(limit=1)
    assert limiter.allow("a") is True
    set_time(WINDOW)
    assert limiter.allow("a") is True


def test_exact_boundary_with_nonzero_start_and_fractional_window() -> None:
    limiter, set_time = make_limiter(limit=1, window=2.5, start=100.5)
    assert limiter.allow("a") is True
    set_time(102.9)
    assert limiter.allow("a") is False
    set_time(103.0)
    assert limiter.allow("a") is True


# --- Denied requests do not count ---------------------------------------------

def test_denied_requests_do_not_extend_the_block() -> None:
    limiter, set_time = make_limiter(limit=1)
    assert limiter.allow("a") is True
    set_time(5.0)
    assert limiter.allow("a") is False  # if this counted, "a" would be blocked until 15
    set_time(10.0)
    assert limiter.allow("a") is True


def test_hammering_client_gets_through_once_window_clears() -> None:
    limiter, set_time = make_limiter()
    for _ in range(LIMIT):
        limiter.allow("a")
    for i in range(1000):
        set_time(i * 0.009)  # 1000 denied attempts spread over 0..9 seconds
        assert limiter.allow("a") is False
    set_time(WINDOW)
    assert limiter.allow("a") is True


# --- Eviction and memory reclamation ------------------------------------------
# These inspect the private _clients map: eviction is invisible through allow().
```

```
def test_idle_clients_are_evicted_on_next_allow() -> None:
    limiter, set_time = make_limiter()
    for i in range(10_000):
        limiter.allow(f"one-time-{i}")
    assert len(limiter._clients) == 10_000

    set_time(WINDOW)
    limiter.allow("newcomer")
    assert list(limiter._clients) == ["newcomer"]


def test_evicted_client_memory_is_actually_freed() -> None:
    limiter, set_time = make_limiter()
    limiter.allow("old")
    old_log_ref = weakref.ref(limiter._clients["old"])

    set_time(WINDOW)
    limiter.allow("new")
    gc.collect()  # not needed on CPython (refcounting), kept for other interpreters
    assert old_log_ref() is None


def test_eviction_is_lazy_and_happens_inside_allow() -> None:
    limiter, set_time = make_limiter()
    limiter.allow("a")
    set_time(WINDOW * 100)
    assert "a" in limiter._clients  # nothing runs until allow() is called
    limiter.allow("b")
    assert "a" not in limiter._clients


def test_clients_with_any_timestamp_in_window_are_kept() -> None:
    limiter, set_time = make_limiter()
    set_time(0.0)
    limiter.allow("a")
    set_time(8.0)
    limiter.allow("a")
    set_time(10.0)
    limiter.allow("b")
    assert "a" in limiter._clients  # its t=8 request is still in the window


def test_client_log_is_trimmed_to_in_window_timestamps() -> None:
    limiter, set_time = make_limiter()
    for t in (0.0, 1.0, 8.0):
        set_time(t)
        limiter.allow("a")
    set_time(10.5)
    limiter.allow("a")
    assert list(limiter._clients["a"]) == [8.0, 10.5]


def test_denied_request_does_not_keep_client_alive() -> None:
    limiter, set_time = make_limiter(limit=1)
    limiter.allow("a")          # t=0, accepted
    set_time(5.0)
    limiter.allow("a")          # denied: adds nothing
    set_time(10.0)
    limiter.allow("b")
    assert "a" not in limiter._clients


def test_recently_active_client_is_not_stuck_behind_idle_one() -> None:
    # Guards the ordering invariant: a client that becomes active again must move
```

```
    # to the back, otherwise it blocks eviction of idle clients queued behind it.
    limiter, set_time = make_limiter()
    set_time(0.0)
    limiter.allow("a")
    set_time(1.0)
    limiter.allow("b")
    set_time(5.0)
    limiter.allow("a")          # a's newest request is now later than b's
    set_time(11.0)
    limiter.allow("c")
    assert list(limiter._clients) == ["a", "c"]  # b evicted, a kept


# --- Construction and typing -----------------------------------------------

@pytest.mark.parametrize(
    ("limit", "window"),
    [(0, WINDOW), (-1, WINDOW), (LIMIT, 0.0), (LIMIT, -1.0)],
)
def test_invalid_configuration_raises(limit: int, window: float) -> None:
    with pytest.raises(ValueError):
        SlidingWindowLimiter(limit, window)


def test_satisfies_rate_limiter_protocol() -> None:
    limiter: RateLimiter = SlidingWindowLimiter(LIMIT, WINDOW)  # checked by mypy
    assert limiter.allow("a") is True
```

Then the implementation:

rate_limiter.py

```
import time
from collections import OrderedDict, deque
from collections.abc import Callable
from typing import Protocol


class RateLimiter(Protocol):
    """Anything with this method can stand in for a rate limiter."""

    def allow(self, client_id: str) -> bool: ...


class SlidingWindowLimiter:
    """At most `limit` accepted requests per client in any rolling `window` seconds.

    A request made exactly `window` seconds ago is outside the window. Denied
    requests are not recorded. Not thread-safe.

    Idle clients are evicted inside allow(). Clients are kept in order of their
    most recent accepted request, so the idle ones are always at the front and
    each call only looks at the clients it removes, plus one. That makes
    eviction amortized O(1) per call, even with millions of clients.
    """

    def __init__(
        self,
        limit: int,
        window: float,
        clock: Callable[[], float] = time.monotonic,
    ) -> None:
        if limit < 1:
            raise ValueError("limit must be at least 1")
```

```
        if window <= 0:
            raise ValueError("window must be positive")
        self._limit = limit
        self._window = window
        self._clock = clock
        # client_id -> timestamps of accepted requests, oldest first. Every log in
        # here is non-empty, and clients are ordered by their newest timestamp.
        self._clients: OrderedDict[str, deque[float]] = OrderedDict()

    def allow(self, client_id: str) -> bool:
        now = self._clock()
        self._evict_idle_clients(now)

        log = self._clients.get(client_id)
        if log is None:
            log = deque()
        else:
            while now - log[0] >= self._window:
                log.popleft()

        if len(log) >= self._limit:
            return False

        log.append(now)
        self._clients[client_id] = log
        self._clients.move_to_end(client_id)
        return True

    def _evict_idle_clients(self, now: float) -> None:
        """Drop clients whose newest accepted request has left the window."""
        while self._clients:
            client_id, log = next(iter(self._clients.items()))
            if now - log[-1] < self._window:
                return
            del self._clients[client_id]
```

The tests are in `test_rate_limiter.py` and the implementation is in `rate_limiter.py`.

How eviction stays cheap with millions of clients. Clients are stored in an `OrderedDict`, ordered by their most recent *accepted* request. Every accepted request moves its client to the back. Because the clock never goes backwards, the front of the dict always holds the client that has been idle longest.

On each `allow()` call, the limiter removes clients from the front until it reaches one that's still active. Each call only examines the clients it removes, plus one more, so the cost is amortized O(1) per call. Scanning every client on every call would be O(clients).

Denied requests don't add a timestamp, so they don't keep a client alive, and one test checks exactly that. Another test covers the ordering rule itself: if a client became active again and stayed at the front, it would block eviction of idle clients behind it.

Memory tests. Eviction can't be seen through `allow()`, so these tests look at the private `_clients` dict. The strongest one takes a `weakref` to an evicted client's deque and asserts the object is actually freed, not just removed from the dict.

Things to flag:

- **I added argument validation you didn't specify.** `limit < 1` and `window <= 0` raise `ValueError`. Without it, a window of 0 would silently allow unlimited requests. Tell me if you'd rather accept those values, for example treating `limit=0` as "deny everything."
- **The ordering relies on a clock that never goes backwards.** `time.monotonic` guarantees that. An injected clock that jumps backwards would break the ordering, and eviction could stop early. I

didn't add a guard for that.

- **The boundary check compares elapsed time directly.** The code checks `now - ts >= window` rather than precomputing `cutoff = now - window`. That's a literal translation of your rule, and it avoids a separate floating-point rounding step that could flip results at the exact boundary.
- **Eviction is lazy.** Idle clients are only removed when *some* client calls `allow()`. If traffic stops completely, the last batch of client data stays in memory until the next call. A test documents this.
- **Memory returns to the allocator, but the dict itself may not shrink right away.** Evicted deques are freed immediately. CPython only shrinks a dict's internal hash table when new entries are inserted, so after a huge spike the table may keep its allocated size until traffic resumes.
- **Worst case per active client is still O(N) timestamps.** That's the tradeoff we accepted for exact rolling windows.
- **The Protocol is only a type annotation.** A future `TokenBucketLimiter` just needs an `allow(self, client_id: str) -> bool` method. It doesn't inherit from anything, and callers annotated as `RateLimiter` won't need to change.

I haven't run the tests.

User — prompt 7

build a minimal pipeline that applies an ordered list of text transforms to a string. include these transforms:

1. trim whitespace
2. collapse repeated spaces
3. capitalize the first letter of each sentence
4. two creative transforms of your choice.

ground rules:

- NO classes. a simple function taking a string and a list of `Callable[[str], str]` is perfect.
- use precise python type hinting.
- write a pytest suite.
- include one test specifically proving that the order of transforms changes the final output.
- for your sentence capitalization transform, write tests for these specific inputs and make sure they pass: `"hello. e.g. this is a test"`, `"ask dr. smith"`, and `"version 3.5 is out"`.

spit out the tests first, then the code.

Claude

Tests first:

test_text_pipeline.py

```
from collections.abc import Callable

import pytest

from text_pipeline import (
    capitalize_sentences,
    collapse_spaces,
    make_truncator,
    run_pipeline,
    tighten_punctuation,
    trim,
)


# --- run_pipeline ------------------------------------------------------------

def test_empty_pipeline_returns_input_unchanged() -> None:
    assert run_pipeline("  as is  ", []) == "  as is  "


def test_single_transform() -> None:
    assert run_pipeline("  hi  ", [trim]) == "hi"


def test_accepts_any_sequence_and_plain_lambdas() -> None:
    assert run_pipeline("abc", (str.upper, lambda s: s + "!")) == "ABC!"


def test_transforms_run_in_list_order() -> None:
    calls: list[str] = []

    def record(name: str) -> Callable[[str], str]:
```

```
        def transform(text: str) -> str:
            calls.append(name)
            return text
        return transform

    run_pipeline("x", [record("first"), record("second"), record("third")])
    assert calls == ["first", "second", "third"]


def test_order_of_transforms_changes_the_output() -> None:
    text = "   hello world"
    truncate = make_truncator(8)

    trim_first = run_pipeline(text, [trim, truncate])
    truncate_first = run_pipeline(text, [truncate, trim])

    assert trim_first == "hello w…"      # trim, then keep 7 chars + ellipsis
    assert truncate_first == "hell…"     # leading spaces used up the budget
    assert trim_first != truncate_first


def test_full_cleanup_pipeline() -> None:
    messy = "  hello  , world .   this is   dr. who !  "
    cleanup = [trim, collapse_spaces, tighten_punctuation, capitalize_sentences]
    assert run_pipeline(messy, cleanup) == "Hello, world. This is dr. who!"


# --- trim ------------------------------------------------------------------

@pytest.mark.parametrize(
    ("text", "expected"),
    [
        ("  hi  ", "hi"),
        ("\t\nhi\n\t", "hi"),
        ("a  b", "a  b"),   # interior whitespace untouched
        ("", ""),
        ("   ", ""),
    ],
)
def test_trim(text: str, expected: str) -> None:
    assert trim(text) == expected


# --- collapse_spaces -------------------------------------------------------

@pytest.mark.parametrize(
    ("text", "expected"),
    [
        ("a  b", "a b"),
        ("a      b   c", "a b c"),
        ("a b", "a b"),
        ("  a  ", " a "),        # collapses runs, does not trim
        ("a\t\tb", "a\t\tb"),    # only spaces, per spec; tabs untouched
        ("a\n\nb", "a\n\nb"),    # newlines untouched
        ("", ""),
    ],
)
def test_collapse_spaces(text: str, expected: str) -> None:
    assert collapse_spaces(text) == expected


# --- capitalize_sentences: required inputs ---------------------------------

def test_capitalize_abbreviation_does_not_end_sentence() -> None:
    # "hello." ends a sentence, so "e.g." starts one and becomes "E.g.".
```

```
    # "e.g." is an abbreviation, so "this" is NOT capitalized.
    assert capitalize_sentences("hello. e.g. this is a test") == "Hello. E.g. this is a test"


def test_capitalize_title_abbreviation() -> None:
    # Only sentence starts are capitalized; "dr." itself is left alone.
    assert capitalize_sentences("ask dr. smith") == "Ask dr. smith"


def test_capitalize_decimal_number_is_not_sentence_end() -> None:
    assert capitalize_sentences("version 3.5 is out") == "Version 3.5 is out"


# --- capitalize_sentences: other behavior ------------------------------------

@pytest.mark.parametrize(
    ("text", "expected"),
    [
        ("hello. world", "Hello. World"),
        ("wow! really? yes.", "Wow! Really? Yes."),
        ("one.\ntwo", "One.\nTwo"),                  # any whitespace separates sentences
        ("  hello. world", "  Hello. World"),        # leading whitespace preserved
        ("hello. NASA rocks", "Hello. NASA rocks"),  # never lowercases anything
        ("Already. Done", "Already. Done"),
        ("i.e. yes", "I.e. yes"),
        ("see Dr. smith", "See Dr. smith"),          # abbreviation match is case-insensitive
        ("it costs 3.5. buy it", "It costs 3.5. Buy it"),
        ("3 cats. dogs too", "3 cats. Dogs too"),    # non-letter first char is left as is
        ("apples, etc. then more", "Apples, etc. Then more"),  # "etc." counts as a sentence end
        ("", ""),
        ("   ", "   "),
    ],
)
def test_capitalize_sentences(text: str, expected: str) -> None:
    assert capitalize_sentences(text) == expected


# --- tighten_punctuation -----------------------------------------------------

@pytest.mark.parametrize(
    ("text", "expected"),
    [
        ("hello , world !", "hello, world!"),
        ("wait   ?", "wait?"),
        ("a ; b : c", "a; b: c"),
        ("fine, already.", "fine, already."),
        ("line\n.", "line\n."),   # only spaces are removed, not newlines
        ("", ""),
    ],
)
def test_tighten_punctuation(text: str, expected: str) -> None:
    assert tighten_punctuation(text) == expected


# --- make_truncator ----------------------------------------------------------

@pytest.mark.parametrize(
    ("max_chars", "text", "expected"),
    [
        (10, "short", "short"),
        (5, "exact", "exact"),           # exactly at the limit: untouched
        (5, "toolong", "tool…"),
        (1, "abc", "…"),
        (3, "", ""),
    ],
```

```
)
def test_truncator(max_chars: int, text: str, expected: str) -> None:
    result = make_truncator(max_chars)(text)
    assert result == expected
    assert len(result) <= max_chars


@pytest.mark.parametrize("max_chars", [0, -1])
def test_truncator_rejects_non_positive_limit(max_chars: int) -> None:
    with pytest.raises(ValueError):
        make_truncator(max_chars)
```

That import at the bottom is sloppy. Let me move it to the top where it belongs. *(The file above is the corrected version, with the* `Callable` *import at the top.)*

Now the code:

text_pipeline.py

```
import re
from collections.abc import Callable, Sequence

Transform = Callable[[str], str]

# Words ending in "." that do not end a sentence. Compared case-insensitively.
# "etc." is deliberately absent: it ends sentences too often to guess.
ABBREVIATIONS: frozenset[str] = frozenset(
    {"mr.", "mrs.", "ms.", "dr.", "prof.", "st.", "jr.", "sr.", "vs.", "e.g.", "i.e."}
)


def run_pipeline(text: str, transforms: Sequence[Transform]) -> str:
    """Apply each transform to the text, in order."""
    for transform in transforms:
        text = transform(text)
    return text


def trim(text: str) -> str:
    """Remove leading and trailing whitespace."""
    return text.strip()


def collapse_spaces(text: str) -> str:
    """Replace runs of two or more spaces with one space. Tabs and newlines are untouched."""
    return re.sub(r" {2,}", " ", text)


def capitalize_sentences(text: str) -> str:
    """Uppercase the first character of each sentence, leaving everything else as is.

    A sentence starts at the beginning of the text and after any word ending in
    ".", "!" or "?" that is followed by whitespace, unless that word is in
    ABBREVIATIONS. Because a sentence end needs whitespace after it, "3.5" is
    never split.
    """
    parts = re.split(r"(\s+)", text)  # words and the whitespace between them
    capitalize_next = True
    for i, part in enumerate(parts):
        if not part or part.isspace():
            continue
        if capitalize_next:
            parts[i] = part[0].upper() + part[1:]
        capitalize_next = part.endswith((".", "!", "?")) and part.lower() not in ABBREVIATIONS
```

```
    return "".join(parts)


def tighten_punctuation(text: str) -> str:
    """Remove spaces before , . ! ? ; and : ("hello , world !" -> "hello, world!")."""
    return re.sub(r" +([,.!?;:])", r"\1", text)


def make_truncator(max_chars: int) -> Transform:
    """Return a transform that cuts text to at most max_chars, ending in "…" if cut."""
    if max_chars < 1:
        raise ValueError("max_chars must be at least 1")

    def truncate(text: str) -> str:
        if len(text) <= max_chars:
            return text
        return text[: max_chars - 1] + "…"

    return truncate
```

The pipeline is one loop: `run_pipeline(text, transforms)` passes the output of each transform into the next. The two creative transforms are `tighten_punctuation`, which turns `"hello , world !"` into `"hello, world!"`, and `make_truncator(n)`. The truncator is a small factory that returns a transform, so a configurable transform still fits the `Callable[[str], str]` shape without needing a class.

The order test runs `" hello world"` through trim and an 8-character truncator in both orders. Trimming first gives `"hello w…"`. Truncating first gives `"hell…"`, because the leading spaces used up part of the 8-character budget.

The three required capitalization inputs. You didn't give expected outputs, so the tests assert results I chose. Please confirm them:

Input	My expected output	The reasoning
`"hello. e.g. this is a test"`	`"Hello. E.g. this is a test"`	`hello.` ends a sentence, so `e.g.` starts one and is capitalized. `e.g.` is an abbreviation, so `this` isn't capitalized. If you'd rather keep `e.g.` lowercase, that's a one-line change.
`"ask dr. smith"`	`"Ask dr. smith"`	The transform only capitalizes sentence starts. It doesn't fix proper nouns or titles, so it doesn't produce `Dr. Smith`.
`"version 3.5 is out"`	`"Version 3.5 is out"`	A sentence end needs whitespace after the punctuation, and `3.5` has none.

Known limits of the capitalization rule:

- **It depends on a fixed abbreviation list.** A real sentence ending in one, like `"I saw the dr. he waved"`, won't capitalize `he`. I left `etc.` out of the list because it ends sentences often. A test documents that choice.
- **Only the very first character of a word is uppercased.** So `'"hello." she said'` stays lowercase, because the first character is a quote mark. I did it this way so something like `3rd` never becomes `3Rd`.

Other choices:

- `collapse_spaces` **only touches spaces,** matching your wording. Tabs and newlines are unchanged, and runs at the edges collapse to one space rather than being removed. Tell me if "spaces" should mean all whitespace.
- `tighten_punctuation` **also removes the space in emoticons,** so `"hi :)"` becomes `"hi:)"`.
- **The truncator counts code points, not visible characters,** so it can split an emoji or an accented letter made of several code points. The ellipsis counts toward the limit, and a limit below 1 raises `ValueError`.

I haven't run the tests.